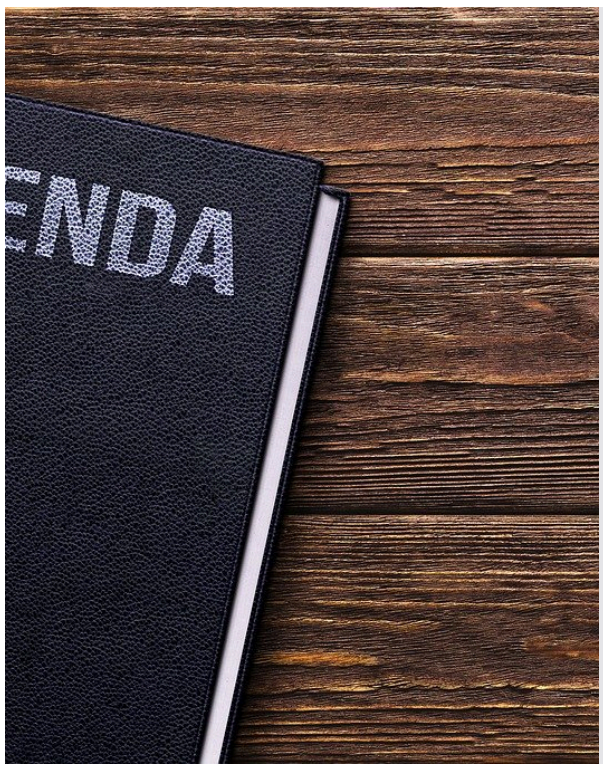
UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

# Introduction to Deep Learning

## Foundations of Machine Learning



# Agenda




- 2.1 Machine learning tasks and terminology
- 2.2 Training and generalization
- 2.3 Example methods
- 2.4 Evaluation

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

# 2.1 Machine Learning Tasks



# Definition and Main Goals

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

- Machine learning is concerned with “statistical algorithms that can learn from data and generalize to unseen data, and thus perform tasks without explicit instructions” – Wikipedia, cf. A. Samuel (1959)

- Most common goal: “Learn” from historical data to make predictions about the future

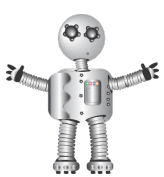
- Alternative goal: Understand more about the domain by learning the structure of the given data

# Running Example: Bot Detection

Is a new user registration a human or a bot?
Learn from past registrations with known answers



**or**



| ID | OS | User-Agent Browser | Email-TLD domain | Time for registration (in s) | Posts/day | Bot? |
|---|---|---|---|---|---|---|
| 1 | Windows | Chrome | .de | 17 | 4 | Yes |
| 2 | Linux | Firefox | .de | 412 | 2 | No |
| 3 | Windows | Firefox | .com | 538 | 0.5 | No |
| 4 | Linux | Chrome | .fr | 392 | 12 | No |
| 5 | Windows | Chrome | .com | 129 | 1 | No |
| 6 | MacOS | Safari | .de | 245 | 3 | No |
| 7 | Linux | Firefox | .com | 25 | 1 | No |
| 8 | Unknown | Firefox | .es | 540 | 3.5 | No |
| 9 | Windows | Chrome | .com | 58 | 2 | No |
| 10 | Linux | Firefox | .de | 89 | 7 | No |
| 11 | Linux | Chrome | .fr | 6 | 145 | Yes |
| 12 | Windows | Chrome | .com | 572 | 2 | Yes |
| 13 | Linux | Safari | .gov | 150 | 5 | No |
| 14 | Linux | Safari | .id | 98 | 0.3 | No |
| 15 | Windows | Chrome | .com | 159 | 2 | No |
| … | … | … | … | … | … | … |
| 1001 | Windows | Firefox | .com | 510 | 1 | ? |
| 1002 | Linux | Chrome | .id | 210 | 145 | ? |
| 1003 | MacOS | Safari | .com | 150 | 1.5 | ? |

**Training**

**Prediction**

# Terminology

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

- A row
  - is called a (data) instance (also: sample, example)
  - describes all the known properties of one entity
  - Rows should be independent of each other
    (and come from the same distribution → *i.i.d.* assumption)

- A column
  - is called a feature (sometimes attribute, or property)
  - Gives information on one property for all instances
  - Columns can be dependent on each other

# Notation

| | |
|---|---|
| Dataset with $n$ instances | $\mathcal{D} = \{(\boldsymbol{x}_1, y_1), ..., (\boldsymbol{x}_n, y_n)\}$ |
| Feature vector of instance $i$ (one row) | $\boldsymbol{x}_i = (x_{i1}, ..., x_{id})$, $d$ features |
| Label / target of instance $i$ | $y_i$ (class or number) |
| Model with parameters $\boldsymbol{w}$ | $f_{\boldsymbol{w}}$ |
| Prediction | $\hat{y}_i = f_{\boldsymbol{w}}(\boldsymbol{x}_i)$ |
| Loss of a single prediction | $\ell(\hat{y}_i, y_i)$ |

*Bot example:* $\boldsymbol{x}_1 = (\text{Windows}, \text{Chrome}, \text{.de}, 17, 4)$, $y_1 = \text{Yes}$, $n = 15$, $d = 5$

# Machine Learning Tasks by Training Feedback

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

- **Unsupervised** Machine Learning:
  - No prediction of a specific thing
  - Learn useful properties about the structure of the features
  - Learn model parameters using dataset without labels
- **Supervised** Machine Learning:
  - There are one (or more) specific target variables to predict
  - Learn model parameters from feature-label pairs
  - Training examples include the correct answer (label)
- **Self-Supervised** Machine Learning:
  - Labels are created automatically from the data itself (data-data pairs)
  - Example: build a model to complete a sentence
    → pretraining of LLMs: predict the next token
- **Reinforcement Learning**
  - Learn model parameters using exploration and sparse rewards
  - Example: board game: result is only known at the end
    → RLHF and reasoning models (lecture 1)

# Unsupervised Machine Learning

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

- Learn the structure of the data $x_1, ..., x_n$ – no labels $y$ given

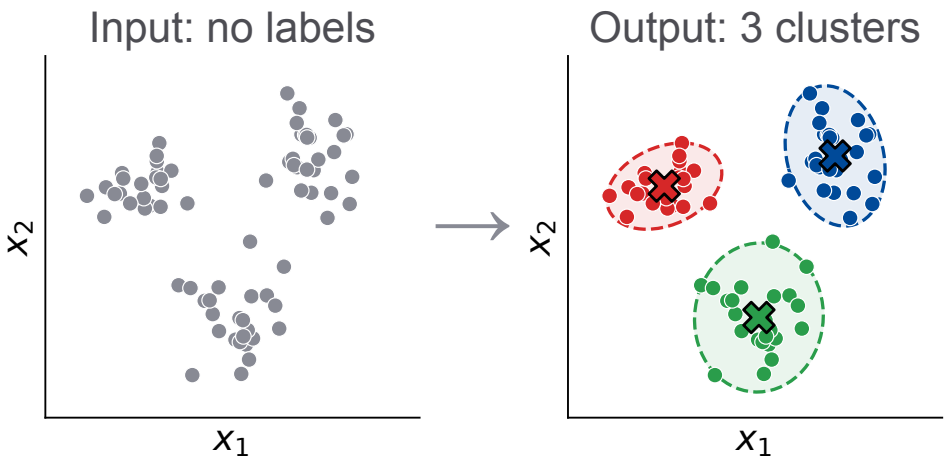
- **Clustering:** find groups of similar instances
  e.g., customer segments, groups of similar gene sequences
- **Dimensionality reduction:** find a compact representation of the data
  e.g., PCA; in deep learning: autoencoders, learned embeddings
- **Density estimation / generative models:** learn the distribution $p(x)$
  e.g., anomaly detection; in deep learning: image and text generation

- Next: clustering as the classical example

# Clustering

- Identification of a finite set of *clusters* in the data (= categories, “classes”, groups)
- Same cluster: as *similar* as possible
- Different clusters: as *dissimilar* as possible
- *Unsupervised*: no groups given

- Examples:
  - Group website users by behavior
  - Groups with similar political opinions
  - Gene sequences with similar expressions
  - Stocks with similar behavior



# Similarity?









# Similarity?









→ Evaluation is difficult!

Photos: pixabay.com

# Supervised Machine Learning Tasks

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

- Output can be discrete, a real number, or more complex:
  - **Classification**: Output is prediction of a class (class or probability)
  - **Multiclass classification**: Choose between more than 2 classes
  - **Multi-label classification**: Predict multiple labels at the same time
  - **Regression**: Output is a number
  - **Structured prediction**: Output is more complex
    (e.g., a sequence, a segmentation mask, a parse tree)

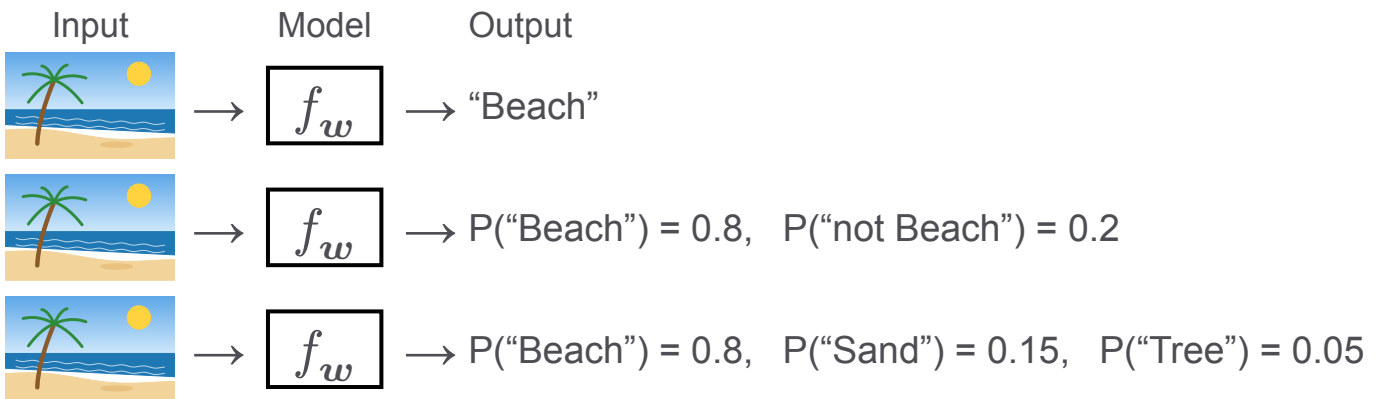
- For classification, the variable that is predicted is the “label”

# Quiz: Which Task Is It?

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

1. Detect spam e-mails, using e-mails that users marked as spam / not spam
2. Group customers by their purchase behavior
3. Predict tomorrow's electricity demand in MWh
4. Pretrain a language model on web text by predicting the next word
5. Teach a robot to walk; it gets a reward for the distance covered
6. Mark all pixels that belong to a tumor in an MRI scan

# Hard vs. Soft Classification




- Soft classification also gives a *confidence*: “My prediction is that soccer team A wins with 80% probability” – would I take that bet?
- Good probabilities are *calibrated*: of all predictions made with 80%, about 80% should be correct
- Later on: *log-likelihood*, *log-odds*, cross-entropy

Slide idea: A. Geiger, “Deep Learning”, 1.3 Machine Learning Basics

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

# 2.2 Training and Generalization



# Supervised Machine Learning Phases

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

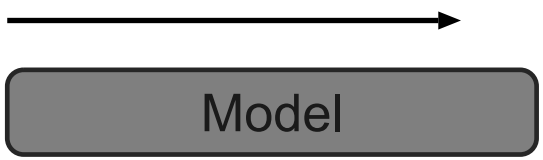
- **Learning:**
  - For training, we choose a type/class of models
  - During training, we learn the concrete model $f_{\boldsymbol{w}}$ of this model type that is in line with the training data
  - Models are often functions with free **parameters** $\boldsymbol{w}$
  - We learn how to choose the parameters such that the model makes good predictions on the training data
    ➔ Loss functions
  - **Hyperparameters** are set by the user and determine the learning procedure (not learned)
- **Inference:**
  - Make novel predictions: $\hat{y} = f_{\boldsymbol{w}}(\boldsymbol{x})$

# Models in ML

| ID | OS | User-Agent Browser | Email-TLD domain | Time for registration (in s) | Posts/day | Bot? |
|---|---|---|---|---|---|---|
| 1 | Windows | Chrome | .de | 17 | 4 | Yes |
| 2 | Linux | Firefox | .de | 412 | 2 | No |
| 3 | Windows | Firefox | .com | 538 | 0.5 | No |
| 4 | Linux | Chrome | .fr | 392 | 12 | No |
| 5 | Windows | Chrome | .com | 129 | 1 | No |
| 6 | MacOS | Safari | .de | 245 | 3 | No |
| 7 | Linux | Firefox | .com | 25 | 1 | No |
| 8 | Unknown | Firefox | .es | 540 | 3.5 | No |
| 9 | Windows | Chrome | .com | 58 | 2 | No |
| 10 | Linux | Firefox | .de | 89 | 7 | No |
| 11 | Linux | Chrome | .fr | 6 | 145 | Yes |
| 12 | Windows | Chrome | .com | 572 | 2 | Yes |
| 13 | Linux | Safari | .gov | 150 | 5 | No |
| 14 | Linux | Safari | .id | 98 | 0.3 | No |
| 15 | Windows | Chrome | .com | 159 | 2 | No |

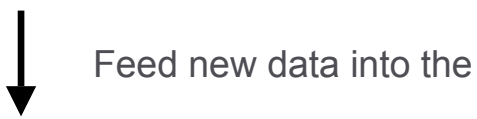
**Training Data**

Algorithm extracts patterns/dependencies



# Models in ML

| | | | | | | |
|---|---|---|---|---|---|---|
| 1001 | Windows | Firefox | .com | 510 | 1 | ? |
| 1002 | Linux | Chrome | .id | 210 | 145 | ? |
| 1003 | MacOS | Safari | .com | 150 | 1.5 | ? |



Feed new data into the model






Model gives predictions

| | | | | | | |
|---|---|---|---|---|---|---|
| 1001 | Windows | Firefox | .com | 510 | 1 | Yes |
| 1002 | Linux | Chrome | .id | 210 | 145 | No |
| 1003 | MacOS | Safari | .com | 150 | 1.5 | Yes |

# Training as Optimization

- A **loss function** (aka error function) measures how good the current parameters are – ideally differentiable
- Training loss = average loss over the training data (*empirical risk*):

$$L(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_{\boldsymbol{w}}(\boldsymbol{x}_i), y_i) \qquad \boldsymbol{w}^* = \arg\min_{\boldsymbol{w}} L(\boldsymbol{w})$$

- Examples for the loss $\ell(\hat{y}, y)$ of a single prediction:

| | | |
|---|---|---|
| Misclassification (0-1 loss): | $\ell = \begin{cases} 1 \text{ if } \hat{y} \neq y \\ 0 \text{ otherwise} \end{cases}$ | not differentiable |
| Squared error (regression): | $\ell = (\hat{y} - y)^2$ | |
| Negative log-likelihood: | $\ell = -\log p_{\boldsymbol{w}}(y \mid \boldsymbol{x})$ | → cross-entropy, next lecture |

- Find the parameters with an optimizer, e.g., gradient descent:
  $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla_{\boldsymbol{w}} L(\boldsymbol{w})$ (learning rate $\eta$ is a hyperparameter; details: next lecture)

# Scoring (Regression)

- Idea:
  - We always start with 1 “point”
  - If $x_1$ is present, we give 1 “point”
  - If $x_2$ is present, we give 3 “points”

| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 2 |
| 0 | 1 | 4 |
| 1 | 1 | 5 |

- How many points we give for each feature are the **parameters**

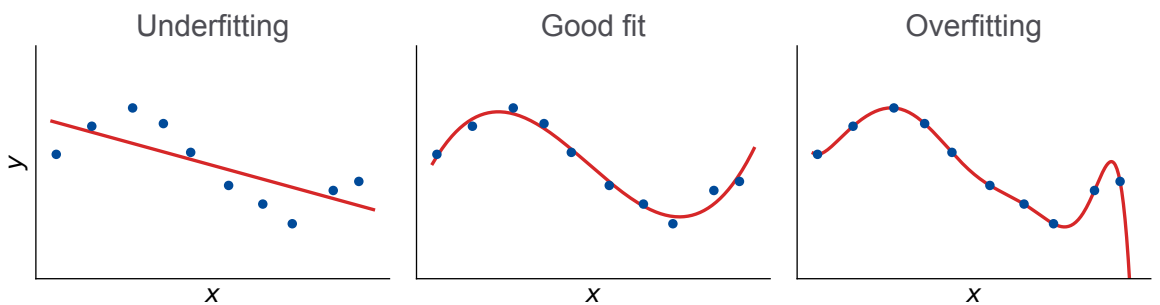
# Scoring (Regression)

- Idea:
  - We always start with 1 “point”
  - If $x_1$ is present, we give 1 “point”
  - If $x_2$ is present, we give 3 “points”
  - If $x_1$ and $x_2$ are present, we give 1 “bonus point”

| $x_1$ | $x_2$ | $y$ |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 0 | 2 |
| 0 | 1 | 4 |
| 1 | 1 | 6 |

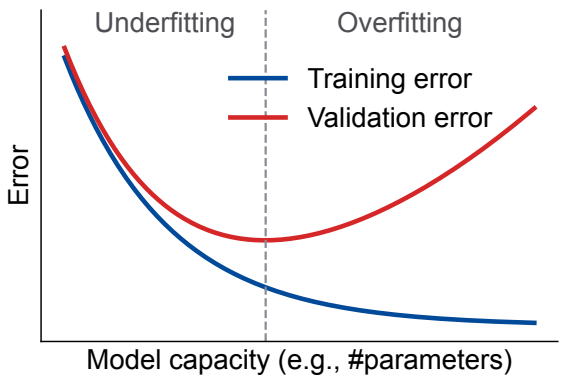
- How many points we give for each feature are the **parameters**
- What we allow to be scored is part of the **hyperparameters**
- With more parameters, we can fit more complex dependencies

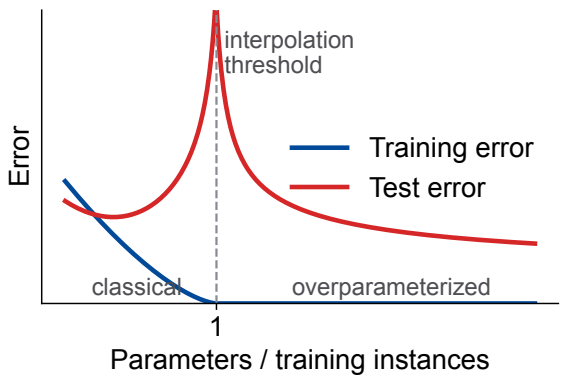
# Overfitting and Underfitting




- **Overfitting**: adapting too precisely to the training data (including its noise)
  – "learning by heart": good on training data, bad on new data
- Classical view: happens if we have too many parameters compared to the number of data instances
- Opposite: **Underfitting** – model too simple to capture the structure of the data

# Generalization and Model Capacity




**Classical view:** choose the model capacity with the lowest error on held-out *validation* data (→ 2.4)

**Deep learning:** "double descent" – very large networks fit the training data perfectly and still generalize well

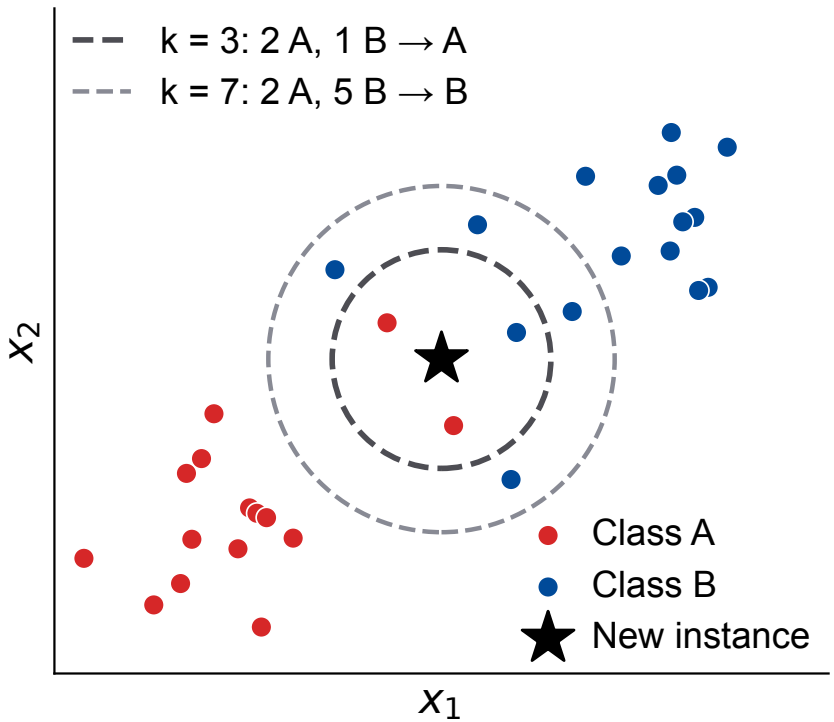
Belkin et al.: Reconciling modern machine-learning practice and the classical bias–variance trade-off. PNAS, 2019
Nakkiran et al.: Deep Double Descent: Where Bigger Models and More Data Hurt. ICLR, 2020 (schematic curves)

# 2.3 Example Methods



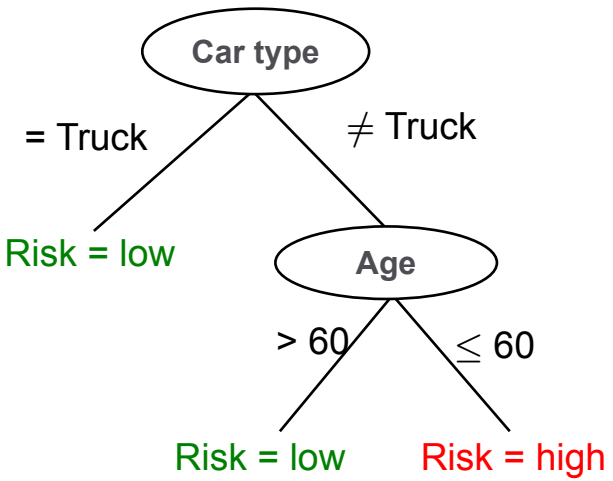
# K-Nearest-Neighbor

- Define distances between data instances
- Specify a value *k* (hyperparameter)
- To classify a new data instance:
  - Search the k most similar instances (k-nearest-neighbors)
  - Select the majority class among those neighbors (“voting”)
  - Option: Weight by distance
    Close-by: High weight
    far-away: Low weight



# Decision Trees

- Each inner node is an attribute
- Each edge is a value (range) of its source node
- Each leaf node is a class label
- Learning: Greedy top-to-bottom
- Prediction:
  Follow the tree from top-to-bottom based on instance values

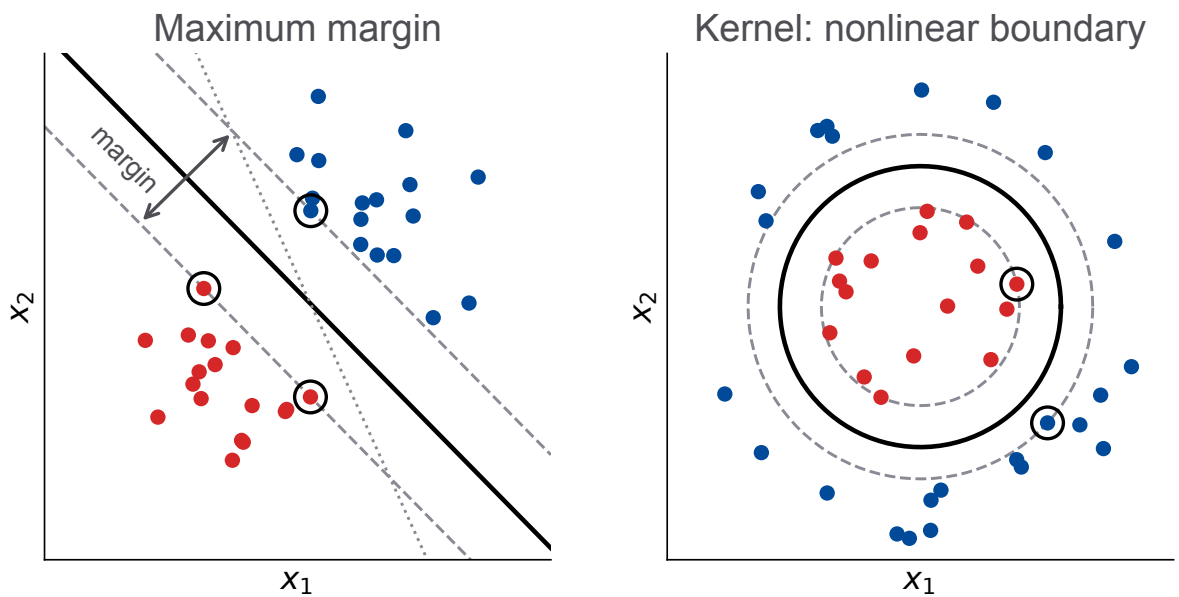



| ID | Age | Car type | Risk |
|---|---|---|---|
| 1 | 23 | Family | high |
| 2 | 17 | Sport | high |
| 3 | 43 | Sport | high |
| 4 | 68 | Family | low |
| 5 | 32 | Truck | low |

# Support Vector Machines

- Data points in $d$-dimensional space
- Find the separating hyperplane with maximal distance to the closest instances (the *margin*)
- These closest instances are the *support vectors* (circled)

- Extensions:
  - Kernel methods: implicit mapping into a higher-dimensional space → nonlinear boundaries
  - Soft margin (slack variables): penalty for instances inside the margin or on the wrong side




Dotted line: also separates the classes, but with a smaller margin

# Logistic Regression

- Linear model plus sigmoid function:

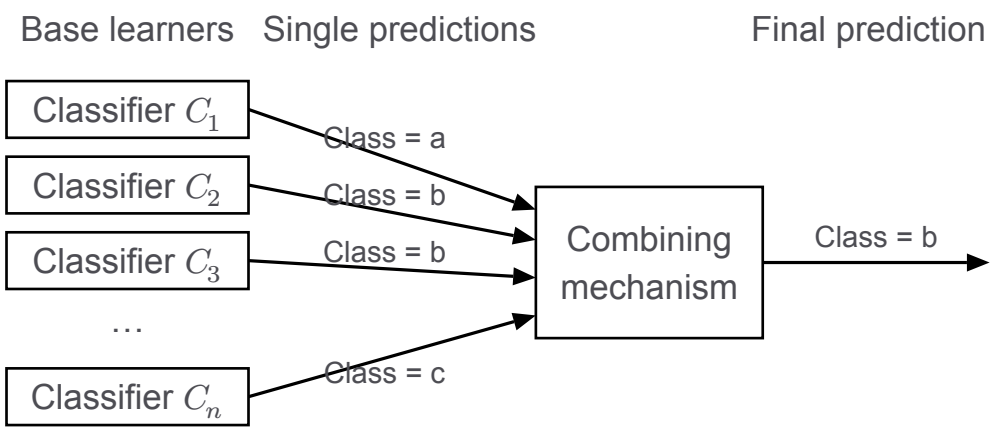
$$\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{x} + b) \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

- Output between 0 and 1: probability of the positive class
  → soft classification
- Equivalent to a single artificial neuron: the simplest neural network

- Details: next lecture (Single-Layer Neural Networks)

# Idea of Ensemble Learning

- Different classifiers make different errors
- Try to combine classifiers to improve classification accuracy!
- Definition Ensemble Learning:
  "An ensemble of classifiers is a set of classifiers whose individual decisions are combined in some way to classify new examples" (Dietterich, 2000)



# Ensemble Learning

| | $i_1$ | $i_2$ | $i_3$ | $i_4$ | $i_5$ | Overall Accuracy |
|---|---|---|---|---|---|---|
| **True class** | **+** | **+** | **+** | **-** | **-** | |
| **Classifier 1** | + | + | + | + | - | 0.8 |
| **Classifier 2** | + | + | - | - | - | 0.8 |
| **Classifier 3** | - | - | + | - | - | 0.6 |
| **Majority voting** | + | + | + | - | - | 1.0 |

# Ensemble Learning Methods

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

- Random Forest: Bagging
  - Build many slightly different trees
  - Subset of instances for each tree (bootstrap sampling)
  - Choose from a subset of attributes at each node
  - Collect predictions for each tree
  - Majority decides
- Boosting (e.g., AdaBoost)
  - Train a simple classifier
  - Identify misclassified instances of this classifier
  - Increase the weight of these instances
  - Iterate: Do this many times
  - Overall classifier: Combination of all previously trained models

# Gradient Boosting and Tabular Data

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

- **Gradient boosting:** each new tree is trained on the remaining errors of the current ensemble (more precisely: the negative gradient of the loss)
- Popular implementations: XGBoost, LightGBM, CatBoost

- For **tabular data** (like our bot example), gradient-boosted trees are still a very strong baseline and often beat deep learning
- Deep learning shines on **unstructured data**: images, text, audio, …
- Recent: tabular foundation models (e.g., TabPFN) catch up, in particular on small datasets

Grinsztajn, Oyallon, Varoquaux: Why do tree-based models still outperform deep learning on typical tabular data? NeurIPS, 2022

Hollmann et al.: Accurate predictions on small data with a tabular foundation model. Nature, 2025

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

# 2.4 Evaluation



# Classifier Evaluation

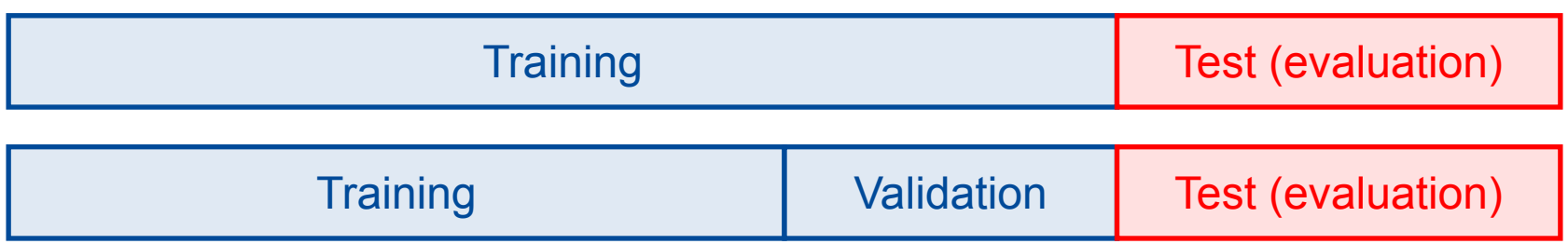
- How do we know how good our classifier will be in the application phase?

- Train-and-Test: divide labeled data in two subsets (e.g., 70%/30%)
  1. Training set to learn a classifier
  2. Test set to evaluate the classifier



# Training, Validation and Test Data

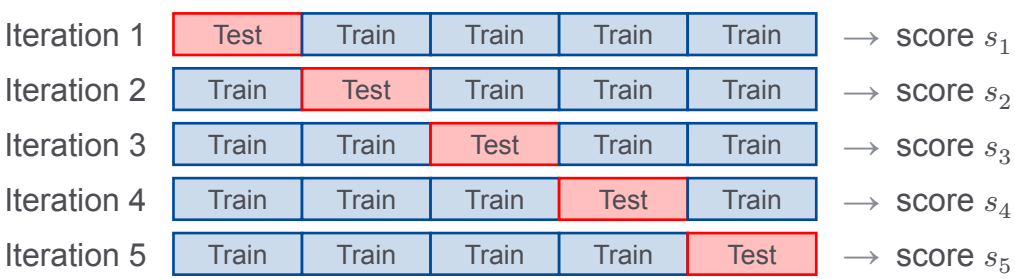
- Split labeled data into training, validation and test data
  - **Training data:** learn the parameters
  - **Validation data:** compare models and choose hyperparameters
  - **Test data:** touched only once at the end, to estimate how well the final model will work on new data



# Cross-Validation

- Simple training-and-test can be subject to random fluctuations
- Better: $k$-fold cross-validation (often $k = 10$)
  - Randomly split labeled data into $k$ parts (folds)
  - Use $k - 1$ parts for training, the last part for testing
  - Repeat $k$ times, each part is used for testing once
  - Average results

- Applied if limited training data is available and training time is reasonable
- More parts: more training data, but higher runtime
- Extreme case: “Leave-one-out” evaluation ($k = n$)
- Rarely used for large deep networks (too expensive)



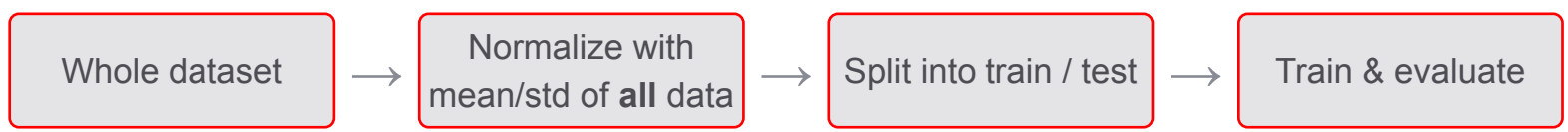

$k = 5$: final estimate

$$\frac{1}{5}\sum_{i=1}^{5} s_i$$

# Information Leakage

- We use information that will not be available at prediction time
- “Leak” information from the validation/test set into the training
  - E.g., pre-processing (e.g., normalization) on the whole dataset

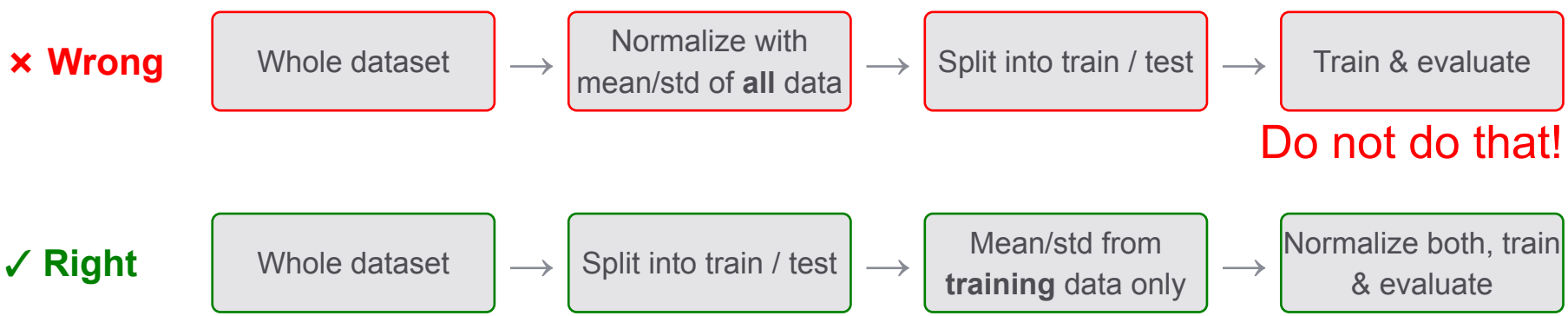
**× Wrong**




Do not do that!

# Information Leakage

- We use information that will not be available at prediction time
- “Leak” information from the validation/test set into the training
  - E.g., pre-processing (e.g., normalization) on the whole dataset



# Quiz: Where Is the Leak?

1. We select the 20 best features using the whole dataset, then run cross-validation
2. Patient study with several scans per patient: we split the *scans* randomly into training and test data
3. Predict loan default using the feature “number of reminder letters sent”
4. We try 50 hyperparameter settings and report the best *test set* score
5. We compute mean and standard deviation on the training data and use them to normalize training and test data

# Leakage in the Age of LLMs

- Benchmarks are published on the web – and web text is used to train LLMs
- **Test set contamination:** the model may have seen the test questions (and answers) during training
- Scores then measure memorization rather than generalization
- Evidence: on a fresh re-creation of a popular math benchmark (GSM8k → GSM1k), several models score clearly lower than on the original

- Countermeasures: new, held-back or continuously updated benchmarks; contamination checks; “canary strings” in benchmark files

Sainz et al.: NLP Evaluation in trouble: On the Need to Measure LLM Data Contamination for each Benchmark. Findings of EMNLP, 2023

Zhang et al.: A Careful Examination of Large Language Model Performance on Grade School Arithmetic, 2024

# When Training and Application Data Differ

- Assumption so far: instances are *i.i.d.* – independent and identically distributed – in training, test and application
- Often violated in practice:
  - **Temporal shift:** bots change their behavior over time → the model gets worse
  - **Covariate shift:** new browsers or e-mail domains that never occurred in training
  - **Dependent instances:** several rows per user or per patient
- Consequences for evaluation:
  - Split by time (train on the past, test on the future) or by group (user, patient)
  - Monitor models after deployment

# Evaluation Measures

Accuracy: $$\frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}$$

Precision: $$\frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$$

Recall: $$\frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

$F_1$-Measure: $$\frac{2 \cdot \mathrm{Precision} \cdot \mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$$

| | | Ground Truth | |
|---|---|---|---|
| | | positive | negative |
| Pred. | positive | True positive (TP) | False positive (FP) |
| | negative | False negative (FN) | True negative (TN) |

# Class Imbalance

- Bots are rare: 10,000 registrations, 100 of them bots
- Trivial classifier “never a bot”:
  accuracy = 99% – but it finds no bot (recall = 0)
- Better classifier: flags 120 users, 80 of them are bots

| | Bot | Human |
|---|---|---|
| Pred. bot | TP = 80 | FP = 40 |
| Pred. human | FN = 20 | TN = 9,860 |

Accuracy = 9,940 / 10,000 = 99.4%
Precision = 80 / 120 $\approx$ 0.67
Recall = 80 / 100 = 0.8
$F_1$ $\approx$ 0.73

- $\rightarrow$ For imbalanced data, report precision / recall / $F_1$ (or PR / ROC curves), not only accuracy

# Regression Evaluation

- Error of instance $i$: $e_i = y_i - \hat{y}_i$
- Mean absolute error

$$\mathrm{MAE}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \frac{1}{n}\sum_{i=1}^{n}|e_i| = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- Mean squared error

$$\mathrm{MSE}(\boldsymbol{y}, \hat{\boldsymbol{y}}) = \frac{1}{n}\sum_{i=1}^{n}e_i^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

# Summary and Outlook

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

**Take-home messages**

- ML learns a model $f_w$ from data instead of explicit rules: supervised, unsupervised, self-supervised, reinforcement learning
- Training = minimizing a loss on the training data; the actual goal is to *generalize* to new data
- Too simple → underfitting, too flexible → overfitting (but: double descent for large networks)
- Classical methods: k-NN, trees, SVMs, ensembles – gradient-boosted trees are strong on tabular data
- Evaluate on held-out data (validation / test, cross-validation), avoid leakage, choose measures that fit the problem

**Next lecture: Single-Layer Neural Networks**

- Linear regression
- Gradient descent, stochastic gradient descent, mini-batches
- Perceptron
- Logistic regression and cross-entropy
- Computation graphs